A. Artifact Appendix

A.1 Abstract

This artifact accompanies the paper **FP8-FLOW-MOE: A CASTING-FREE FP8 RECIPE WITHOUT DOUBLE QUANTIZATION ERROR**.
The artifact includes:

1. **Modified TransformerEngine** — with fused FP8-aware operators (permute-and-padding, scaling-aware FP8 transpose, and FP8 handling logic inside `grouped_linear`).
2. **Modified Megatron-LM** — integrating the FP8-FLOW-MoE training pipeline and fused FP8-aware operators (permute-and-padding, SwiGLU+quantization, FP8 communication), plus fine-grained recomputation for `moe_expert`.
3. **FP8-FLOW-MoE test suite** — benchmark scripts and expected-result checks for reproducing the paper's experiments.

Minimum hardware: NVIDIA Hopper GPUs (FP8-capable, High HBM recommended). Operator micro-benchmarks require 1 GPU; FP8 communication micro-benchmarks require 1–4 nodes (8 GPUs/node); large-scale end-to-end efficiency/ablation/convergence experiments are designed for up to 32 nodes (256 GPUs), depending on the specific configuration.
Key results to reproduce:

- Convergence validation showing no loss degradation under FP8 training (Section 4.1).
- End-to-end training efficiency improvements on DeepSeek-V3/V2 and Qwen3-235B models (Section 4.2).
- End-to-end component ablations on DeepSeek-V3 (671B) under the same setup as Section 4.2 with **full activation checkpointing** (Section 4.3).
- Operator-level speedups for fused permute-and-padding, fused SwiGLU+quantization, scaling-aware FP8 transpose, and FP8 communication (Section 4.4).

A.2 Artifact check-list (meta-information)

- **Algorithm:** Yes. FP8-FLOW-MoE is an end-to-end FP8 mixed-precision training framework for MoE models, including fused operators and FP8 communication strategies.
- **Program:** Modified TransformerEngine, modified Megatron-LM, and custom benchmark scripts (Python + shell). All benchmark scripts are included in this artifact.
- **Compilation:** TransformerEngine must be compiled from source (requires CUDA toolkit).
- **Model:** DeepSeek-V3/V2 and Qwen3-235B model configurations are used for end-to-end evaluation. Model weights are initialized/continued from publicly released checkpoints.
- **Data set:** End-to-end experiments and convergence validation use publicly available datasets. Operator micro-benchmarks use synthetic data (no external datasets required).
- **Run-time environment:** Linux (tested on Ubuntu 24.04.2 LTS). Requires CUDA 12.9+, PyTorch, Triton 3.2.0, DeepEP 1.0.0.5. No root access required for running experiments.
- **Hardware:** NVIDIA Hopper GPUs. Intra-node: NVLink; inter-node: RoCE.
  - Operator micro-benchmarks: 1 GPU.
  - FP8 communication micro-benchmarks: 1–4 nodes (8 GPUs/node).
  - End-to-end efficiency/ablation experiments: designed for multi-node (up to 32 nodes/256 GPUs depending on EP/PP settings).
- **Run-time state:** Micro-benchmarks include warm-up iterations before timing to reduce cold-cache effects. Results may vary due to GPU thermal state and system load.
- **Execution:** Sole-user access during benchmarking is recommended. No special process pinning required. Operator benchmarks complete in minutes; end-to-end experiments can take hours per configuration at scale.
- **Metrics:** Kernel execution time (ms), speedup ratio (fused vs baseline), training throughput (TGS: tokens/GPU/s), peak GPU memory (GB/GPU), convergence loss curves.
- **Output:**
  - Operator micro-benchmarks output CSV files (`perf_data*.csv`), console logs, and optional PDF plots; expected-result checks are included (e.g., aggregated reports).
  - End-to-end experiments output Megatron training logs; throughput is read from logs; peak memory is measured externally.
- **Experiments:** We provide container support and step-by-step reproduction instructions in the repository `git@github.com:021ai/FP8-FLOW-MOE-AE.git` (see `README.md` and `tests/` subdirectories).
- **How much disk space required (approximately)?:** ~10 GB for source code + build artifacts (Megatron-LM + TransformerEngine + tests). End-to-end experiments require additional disk space for datasets and checkpoints (varies by model/dataset).
- **How much time is needed to prepare workflow (approximately)?:** ~2 hour for cloning repos and compiling/installing TransformerEngine (excluding large dataset/checkpoint downloads).
- **How much time is needed to complete experiments (approximately)?:** Operator benchmarks ~1 hour total; end-to-end experiments are several hours per configuration (and depend on cluster scale).
- **Publicly available?:** Yes. Source code is publicly available on GitHub.
- **Code licenses (if publicly available)?:**
- **Workflow frameworks used?:** No special workflow framework; standard Python scripts and shell scripts.
- **Archived?:**

A.3 Description

A.3.1 How to access

The artifact consists of three repositories.

Option 1 (Recommended): Clone our pre-integrated repositories
Strongly recommended: please clone our *pre-integrated* repositories directly. They include our integrated end-to-end test launchers and the component-ablation implementations validated in our environment, which is the most reliable way to reproduce the paper's results.

- `git@github.com:021ai/Megatron-LM-FP8FlowMoe.git`
- `git@github.com:021ai/TransformerEngine-FP8FlowMoe.git`
- `git@github.com:021ai/FP8-FLOW-MOE-AE.git`

Option 2: Apply upstream PRs to official repositories

- **Megatron-LM:**
  Official repository: NVIDIA/Megatron-LM (base commit: `4af25f`)
  Apply our PRs:
  `https://github.com/NVIDIA/Megatron-LM/pull/2763`
  `https://github.com/NVIDIA/Megatron-LM/pull/2764`
- **TransformerEngine:**
  Official repository: NVIDIA/TransformerEngine (base commit: `40c69e7`)
  Apply our PRs:
  `https://github.com/NVIDIA/TransformerEngine/pull/1921`
  `https://github.com/NVIDIA/TransformerEngine/pull/2547`

https://github.com/NVIDIA/TransformerEngine/pull/2144

- **FP8-FLOW-MoE test scripts (our repo):** `git@github.com:021ai/FP8-FLOW-MOE-AE.git`

Approximate disk space after cloning is about 5 GB, excluding build artifacts, checkpoints, and datasets.

A.3.2 Hardware dependencies

- **GPU:** NVIDIA Hopper (FP8-capable; high HBM recommended)
- **Intra-node interconnect:** NVLink
- **Inter-node interconnect:** RoCE, Mellanox ConnectX (`mlx5`)

Minimum requirements by experiment:

Experiment	GPUs required
Convergence validation (Section 4.1)	Multi-GPU; exact scale depends on the launcher and can be smaller than the full 32-node runs.
End-to-end efficiency evaluation (Section 4.2)	Multi-node at scale; up to 32 nodes / 256 GPUs depending on EP/PP settings.
End-to-end component ablations (Section 4.3)	Same setup as Section 4.2.
Operator micro-benchmarks: fused kernels (Sections 4.4.1, 4.4.3, 4.4.4)	1 GPU.
FP8 communication micro-benchmarks (Section 4.4.2)	EP8:1 node; EP16: 2 nodes; EP32:4 nodes.

A.3.3 Software dependencies

Software	Version tested	Notes
OS	Ubuntu 24.04.2 LTS	Other modern Linux distributions should also work.
CUDA Toolkit	12.9.41	Tested version.
Python	3.12	Python $\geq$ 3.10 should also work.
PyTorch	2.7.0a0	Tested with NVIDIA NGC nightly build.
Triton	3.2.0	Typically installed alongside PyTorch.
DeepEP	`1.0.0.5+6519814`	Version 1.2.1+b57e5e2 also tested.
NCCL	CUDA 12.x compatible	Required for multi-GPU and multi-node communication.
GCC/G++	13.3.0	Tested version.

A.3.4 Data sets

Operator micro-benchmarks do not require external datasets because they use synthetic inputs.
Convergence validation, end-to-end efficiency evaluation, and ablations use public datasets, but those datasets are not bundled with the artifact. The latest dataset download and preprocessing instructions are provided in the public artifact repository:`git@github.com:021ai/FP8-FLOW-MOE-AE.git`

A.3.5 Models

End-to-end efficiency evaluation and ablations follow DeepSeek-V2/V3 and Qwen3-235B model configurations. Model weights are not bundled with the artifact; the workflow continues from publicly released checkpoints to keep initialization comparable across methods. The latest checkpoint download instructions are provided in the public artifact repository:`git@github.com:021ai/FP8-FLOW-MOE-AE.git`

A.4 Installation

Assuming the working directory is `/mnt/data/FP8-FLOW-MOE-AE`:

```
# 1. Create and enter working directory
mkdir -p /mnt/data/FP8-FLOW-MOE-AE
cd /mnt/data/FP8-FLOW-MOE-AE

# 2. Clone repositories
git clone  -b release_v1.0 --recursive \
    git@github.com:021ai/Megatron-LM-
        FP8FlowMoe.git
git clone  -b release_v1.0 --recursive \
    git@github.com:021ai/TransformerEngine-
        FP8FlowMoe.git
git clone git@github.com:021ai/FP8-FLOW-MOE-
    AE.git

# 3. Install TransformerEngine from source
pip uninstall -y transformer_engine
cd TransformerEngine-FP8FlowMoe
export NVTE_FRAMEWORK=pytorch
pip3 install --no-build-isolation .
cd ..

# 4. Install Megatron-LM
cd Megatron-LM-FP8FlowMoe
pip install -e .
cd ..
```

A.5 Experiment workflow

This artifact provides four groups of experiments under `FP8-FLOW-MOE-AE`, corresponding to Sections 4.1–4.4 of the paper.

```
tests/
|-- convergence_validation/       # Section 4.1
|-- e2e_efficiency_evaluation/    # Section 4.2
|-- e2e_component_ablations/      # Section 4.3
`-- operator_benchmarks/          # Section 4.4
    |-- fused_permute_and_padding/
    |-- fp8_communication/
    |-- fused_swiglu_and_quantization/
    `-- scaling_aware_fp8_transpose/
```

Common workflow stages are as follows.

1. **Environment preparation.** Install and compile dependencies, including TransformerEngine-FP8FlowMoe, Megatron-LM-FP8FlowMoe, and DeepEP; ensure CUDA, NCCL, and NVSHMEM are configured correctly.
2. **Experiment execution.** Run the corresponding shell launcher or benchmark script.
3. **Result collection.** Operator benchmarks generate `perf_data*.csv` and an aggregated report; end-to-end runs generate Megatron training logs; convergence validation generates TensorBoard event files.
4. **Optional plotting.** Operator benchmarks may include scripts such as `plot_perf.py` to produce paper-ready figures.

Important path customization. End-to-end launchers contain cluster-specific absolute paths for datasets, tokenizers, and checkpoints. Reviewers must replace those paths with local equivalents or parameterize them through environment variables.

Experiment group A: Convergence validation (Section 4.1). Goal: validate that FP8-FLOW-MoE does not degrade convergence relative to BF16.

```
cd tests/convergence_validation
bash run_convergence.sh <bf16|fp8flowmoe>
```

Outputs are written under `$OUTPUT_DIR`, TensorBoard files appear under the run's log directory.

Experiment group B: End-to-end efficiency evaluation (Section 4.2).

- **Goal:** reproduce throughput (TGS: tokens/GPU/s) and peak memory across precision modes and EP/AC settings.
- **Entry script:** `tests/e2e_efficiency_evaluation/run_dlc_e2e.sh`
- **Interface:**

```
cd tests/e2e_efficiency_evaluation
bash run_dlc_e2e.sh <MODEL> <PRECISION> <
    EP> <AC>
```

- **Outputs:** training logs under `${OUTPUT_DIR}` or console; expected values in `expected_output/expected`$results.csv$.
- **Metrics:** TGS from logs (use stable iters 2–6); peak memory measured externally.

Experiment group C: End-to-end component ablations (Section 4.3).

- **Goal:** reproduce component ablations on **DeepSeek-V3 (671B)** under the same setup as Group B with **AC=full**.
- **Entry script:** `tests/e2e_component_ablations/run_dlc_e2e.sh`
- **Interface:**

```
cd tests/e2e_component_ablations
bash run_dlc_e2e.sh <perm_pad|double_quant
    > <ep8|ep16|ep32>
```

- **Outputs:** training logs under `${OUTPUT_DIR}`; expected values in `expected_output/expected_results.csv`.
- **Metrics:** same as Group B.

Experiment group D: Operator-level micro-benchmarks (Section 4.4).

- **Entry points:**
  - benchmarks: `operator_benchmarks/*/bench*.py`
  - One-click runner & reporter: `operator_benchmarks/run_all_benchmarks.py`
- **Outputs:**
  - Per-benchmark: `perf_data*.csv`
  - Aggregated report: `operator_benchmarks/validation_report.txt`

A.6 Evaluation and expected results

This section is a **reproduction/validation checklist**: for each key claim, we provide (1) concrete steps to run, (2) where to find outputs, and (3) success criteria (including acceptable variation).

Feasibility note: Large-scale end-to-end experiments (Sections 4.2–4.3) require substantial Hopper GPU cluster resources. If reviewers do not have such a cluster, they can still fully evaluate Section 4.4 (operator micro-benchmarks) and partially evaluate convergence depending on their available scale.

Claim 1: Convergence validation (Section 4.1)

1. Run

```
cd tests/convergence_validation
bash run_convergence.sh <bf16|fp8flowmoe>
```

2. Inputs that must be provided The script contains hard-coded paths that must be edited: *TOKENIZER_PATH DATASET_FILE*
3. Where to check loss curves Loss is logged via TensorBoard under the run output directory: `${OUTPUT_DIR}/*/tensorboard/events.*`
4. Pass criteria / acceptable variation A successful reproduction should show stable convergence (no divergence / NaNs), and The relative loss error remains consistently below 0.25% (FP8-FLOW-MoE vs BF16) over the compared steps.

Claim 2: End-to-end training efficiency improvements (Section 4.2)

1. Inputs that must be provided the launcher `tests/e2e_efficiency_evaluation/run_dlc_e2e.sh` contains cluster-specific absolute paths that must be edited:

   CPT_PRETRAIN_CHECKPOINT_PATH
   TOKENIZER_PATH
   DATASET_FILE

2. Run

```
cd tests/e2e_efficiency_evaluation
bash run_dlc_e2e.sh <MODEL> <PRECISION> <
    EP> <AC>
```

3. How to read the metrics (TGS & peak memory)

   TGS (tokens/GPU/s): Each run trains for 6 iterations. Use the best TGS among iterations 2–6.

   Example log line:

```
iteration 4/6 | elapsed time per iteration
     (ms): 113993.1 | token throughput per
     GPU (tokens/s/GPU): 1149.8 | ...
```

   Peak memory (GB/GPU): This is not reported in the training logs. Measure peak `memory.used` per GPU via your cluster's GPU monitoring system (e.g., `nvidia-smi`, DCGM, or equivalent) during training.

4. Expected results / acceptable variation

   The full set of expected values (Tables 1–6 in the paper) is provided in the following reference file:

   `e2e_efficiency_evaluation/expected_output/`

   A successful reproduction should satisfy the following:

   - The qualitative trends match the paper: FP8-FLOW-MoE achieves throughput no lower than BF16; larger gains appear at higher EP; and selective recomputation enables higher EP without OOM.
   - Absolute TGS may vary across clusters; a deviation within $\pm$**15%** is acceptable due to hardware and network differences.

Claim 3: Component ablations (Section 4.3)

1. Run

```
cd tests/e2e_component_ablations
bash run_dlc_e2e.sh <perm_pad|double_quant
    > <ep8|ep16|ep32>
```

2. Metrics and judgement Use the same method as Claim 2 to read TGS and peak memory. Acceptable variation is the same as Claim 2.
3. Expected results All ablations run on **DeepSeek-V3 (671B)** with **AC=full**. The full expected values (Table 7 in the paper) are provided in the following reference file:

   `e2e_component_ablations/expected_output/`

   Key qualitative claims to verify:

   - `perm_pad`: disabling fused permute-and-padding reduces throughput by up to ~5% and increases peak memory by approximately 1 GB compared to the non-ablated FP8-FLOW-MoE.
   - `double_quant`: replacing scaling-aware FP8 transpose with DoubleQuant reduces throughput by up to ~18% and increases peak memory by approximately 1 GB depending on EP.

   Acceptable variation: same $\pm$**15%** as Claim 2.

Claim 4: Operator-level speedups (Section 4.4)

1. Run

   Option 1: one-click (recommended)

```
cd tests/operator_benchmarks

python run_all_benchmarks.py
                    # single-GPU benchmarks
     only
python run_all_benchmarks.py --with-comm
      # also run FP8 comm benchmark (
    requires 8 GPUs)
```

   Option 2: run individual sub-experiments

   Each sub-experiment can also be run independently. After running, use `-report-only` to generate the aggregated report.

```
cd tests/operator_benchmarks

# Section 4.4.1 --- Fused Permute and
    Padding (1 GPU)
cd fused_permute_and_padding && python
    bench.py && cd ..

# Section 4.4.2 --- FP8 Communication,
    intra-node (8 GPUs)
cd fp8_communication && python
    bench_intranode.py && cd ..

# Section 4.4.3 --- Fused SwiGLU and
    Quantization (1 GPU)
cd fused_swiglu_and_quantization && python
     bench.py && cd ..

# Section 4.4.4 --- Scaling-aware FP8
    Transpose (1 GPU)
cd scaling_aware_fp8_transpose && python
    bench.py && cd ..

# Generate aggregated report from existing
     results (without comm results)
python run_all_benchmarks.py --report-only
```

2. Expected outputs
   - `tests/operator_benchmarks/*/perf_data*.csv`
   - `tests/operator_benchmarks/validation_report.txt` (also printed to console)
3. Pass criteria / acceptable variation A successful reproduction should satisfy the following:
   - Fused operators and FP8 communication are faster than the corresponding baselines (speedup > 1) across most tested configurations.
   - A deviation within $\pm$**15%** in speedup is acceptable, as long as the qualitative conclusion still holds.

A.7 Experiment customization

Skip this section if the main experiments in Sections 4.1–4.3 can be reproduced successfully on your cluster.
The customization options below are provided for reviewers who lack sufficient cluster resources to run the full paper configurations.

Single-node functional smoke-test (DeepSeek-V2-Lite, 16B)

If your evaluation environment cannot provision the multi-node cluster required for the paper's main experiments (DeepSeek-V3 671B / Qwen3-235B / DeepSeek-V2 236B with EP8–EP32), the script `e2e_efficiency_evaluation/run_dsw_DeepSeek-V2-Lite.sh` provides a lightweight alternative that runs on a single node (8 GPUs), this is just for functional correctness,verifying that all four precision modes launch and complete without errors

Property	Value
Model	DeepSeek-V2-Lite (16B)
Hardware required	1 node, 8 $\times$ GPUs
Parallelism	PP=2, EP=4
Training iterations	6 (configurable via `TRAIN_ITERS`)

Usage:

```
cd tests/e2e_efficiency_evaluation
bash run_dsw_DeepSeek-V2-Lite.sh <CASE>
# CASE: bf16 | blockwise | tensorwise |
    fp8flowmoe
```

Important caveats:

- The throughput (TGS) numbers produced by this script **cannot be compared to any table in the paper**. The model, EP degree, batch size, and node topology all differ from the paper's configurations.
- This script is intended **only** to confirm end-to-end functional correctness (model compiles, training steps run, no NaN/OOM errors) across all four precision modes.
- Before running, update `BASE_DIR`, `TOKENIZER_PATH`, and `DATASET_FILE` inside the script to match your local paths.

A.8 Notes

For the most up-to-date Artifact Evaluation reproduction instructions, please refer to the latest `README.md` in the repository `git@github.com:021ai/FP8-FLOW-MOE-AE.git`.